



Vehiculos

Nombres y apellidos del alumno(a):
Joshua Ontiveros Cabrero

Nombre del docente:
Naranjo Avilez Jesus

Carrera Tecnologías de la información:
Programación orientada a objetos.

Universidad Politécnica de Baja California.
Mexicali Baja California

29/05/2026

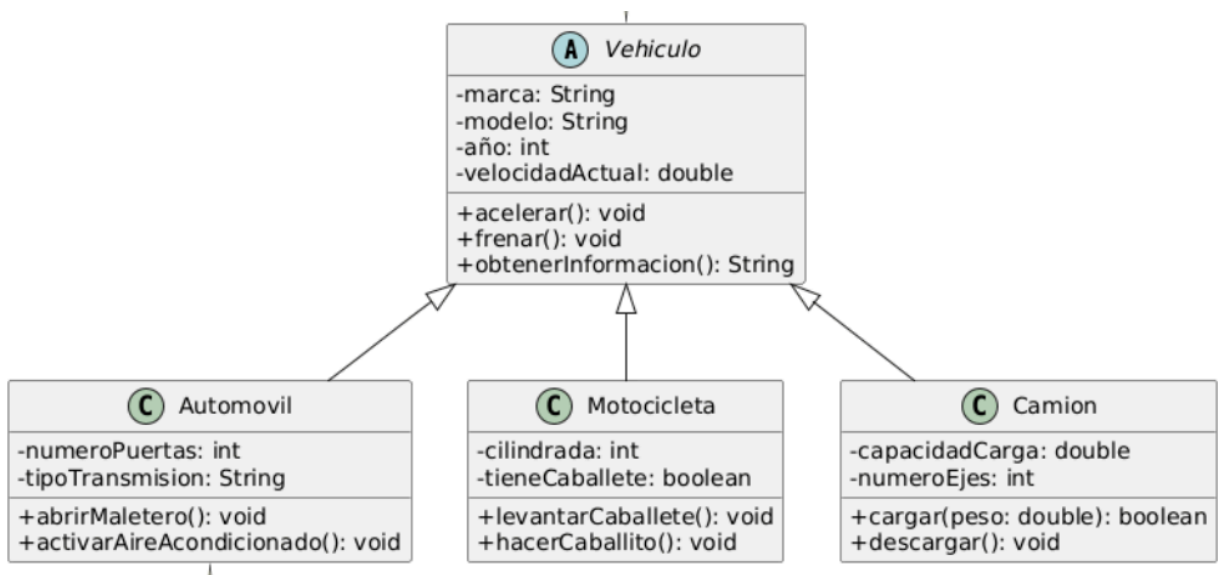
Resumen.

En esta actividad se verán algunas bases fundamentales de la programación orientada a objetos además de los usos básicos de los diagramas tipo UML, que depende de que en este caso, cuales son las características de las clases, sus atributos y sus métodos. Para este caso utilizamos el caso de diversos vehículos. Ya fuera una moto, un carro, un camión de carga.

Introduccion.

Aquello que hace que destaque este programa es el hecho de que está hecho modularmente usando un archivo como la goma que va a unir a todas la cosas por aparte que quisiéramos añadir, lo que le brinda escalabilidad a nuestro programa, dicha goma le llamamos vehículo y después fuimos añadiendo poco a poco las clases que deseáramos que entraran en el ámbito de vehículos. Pero nos estamos adelantando mucho, debemos comenzar por el principio, debemos empezar desde el diagrama UML (Lenguaje Unificado Modelado) el primer paso fue que creamos la clase de Vehículo la cual tendrá dentro de sí misma las funciones comunes que tienen los diversos tipos de vehículos terrestres que vemos en nuestro día a día lo cual hace que no haya redundancias en el trabajo y nos ahorra muchos pasos extras en nuestro trabajo.

Cuenta con algunos atributos los cuales son comunes en todos los vehículos terrestres, ya sea marca, modelo, velocidad actual y año, en el lado de sus funciones tenemos la capacidad de acelerar y frenar. Acto seguido hacemos lo mismo con las demas clases, las cuales llamaremos las clases hijas, estas son dependientes directas de la clase padre que creamos: vehículo. En las clases hijas metemos todas aquellas funciones exclusivas de cada clase, un carro tiene varias puertas, una moto puede hacer Wheelie y un camión puede cargar bastante más peso que un carro.



Desarrollo.

Antes de comenzar, creamos el archivo “vehiculo.py” el cual albergará todas las funciones generales que tienen en común todos nuestros vehículos tal y como lo vimos en el diagrama UML que se mostró en el inicio de la práctica. Necesitamos antes que nada llamar a las herramientas de python para poder crear nuestra clase abstracta

```
#vehiculo.py
from abc import ABC, abstractmethod #(Abstract base classes).
```

Acto seguido vamos a instanciar a la estrella de este programa, la clase “vehiculo”, lo que se hizo fue definir a Vehiculo como una clase y dentro del paréntesis metemos el abstract base class (ABC) haciendo que el Vehiculo funcione como nuestro molde para lo que está por venir.

Dentro de dicha clase tenemos que definir nuestro instanciador, en este caso lo que estamos instanciando sería el propio vehículo, la marca, el modelo, el año y la velocidad actual; estos aspectos son los atributos.

```
class Vehiculo (ABC):
    def __init__(self, marca: str, modelo: str, anyo: int):
        #los guiones bajos representan el modificador privado (-) del diagrama. (Atributos)
        self.__marca = marca
        self.__modelo = modelo
        self.__anyo = anyo
        self.__velocidadActual = 0.0 #inicia en 0 por defecto.
```

Tras esto tenemos que definir los métodos públicos, este primer caso vamos a definir acelerar, la cual incrementara la velocidad del vehículo además funcionando como un velocímetro mostrando la velocidad actual.

```
#Metodos publicos (+). (Funciones)
def acelerar(self):
    self.__velocidadActual += 10.0
    print(f"Acelerando... Velocidad actual: {self.__velocidadActual}km/h")
```

Los restantes son el frenado el cual implementa un if, siendo que si el vehiculo ya esta detenido mostrará que no tiene velocidad, pero si por el contrario si esta moviendose entonces mostrara la resta de dicha función. Además de usar uno muy importante, una función que nos mande a otro lugar la información que tengamos recabada de dicho vehículo

```

def frenar(self):
    if self.__velocidadActual >= 10.0:
        self.__velocidadActual -= 10.0

    else:
        self.__velocidadActual = 0.0
        print(f"Frenando... Velocidad actual: {self.__velocidadActual}km/h")

def obtenerInformacion(self) -> str:
    return f"Vehiculo: {self.__marca} {self.__modelo} ({self.__anyo})"

```

Ya tenemos definida nuestra clase Padre, tiene imbuida todas las funciones que un vehiculo puede tener en general, es hora de ponernos específicos comenzando por la clase “carro”.

```

from vehiculo import Vehiculo

class Automovil(Vehiculo):
    def __init__(self, marca: str, modelo: str, anyo: int, numeroPuertas: int, tipoTransmision: str):
        #llamamos al constructor de la clase padre(Vehiculo)

```

Comenzamos con lo mismo importando, en este caso importamos de la clase vehiculo el objeto Vehiculo para poder obtener la información que requiramos, acto seguido crearemos la clase Automovil y dentro del parentesis indicamos que hereda características de la clase Vehiculo.

Luego de ello definimos nuestro constructor que aparte de recibir la informacion general que tomamos de vehiculo, tambien añadiremos los atributos específicos del vehiculo.

Estos serán los atributos específicos del automovil

```

#atributos especificos del automovil.
self.__numeroPuertas = numeroPuertas
self.__tipoTransmision = tipoTransmision

```

Acto seguido tendremos que definir las funciones específicas del vehiculo, definimos abrir maletero y tambien el activar aire acondicionado puesto que en esto destaca el automovil.

```
def abrirMaletero(self):
    print(f"Abriendo el maletero del auto {self.obtenerInformacion()}....")

def ActivarAireAcondicionado(self):
    print(f"Aire acondicionado acivado; que fresco!")
```

Haremos exactamente lo mismo con una nueva clase la cual llamaremos motocicleta, recordando mandar en el constructor las cosas especificas que haga una motocicleta, en este caso la cilindrada, el caballete y además añadi la variable “patita” con un valor False por defecto, pronto veremos para que funciona. No olvidar, tambien definimos los atributos de la moto.

```
from vehiculo import Vehiculo

class Motocicleta(Vehiculo):
    def __init__(self, marca: str, modelo: str, anyo:int, cilindrada: int, caballete: str, patita: bool):
        super().__init__(marca, modelo, anyo)

        self.__patita = False

        #atributos de la moto
        self.__cilindrada = cilindrada
        self.__caballete = caballete
```

En las funciones especificas definimos la capacidad de hacer caballito y subir o bajar el caballete. Por defecto “patita” la dejamos en false puesto que interpretamos como si el caballete esta arriba, si ya estaba previamente arriba y se intenta subir indicara que ya esta arriba y viceversa

```
def levantarCaballito(self):
    print(f"Haciendo un caballito en la moto...")

def bajarCaballete(self):
    print(f"Bajando el caballete de la moto...")
    if self.__patita == False:
        self.__caballete = "Abajo"

        print(f"Caballete {self.__caballete}")
        self.__patita = True
    else:
        print(f"El caballete ya esta abajo")

def subirCaballete(self):
    print(f"Subiendo el caballete de la moto...")
    if self.__patita == True:
        self.__caballete = "Arriba"

        print(f"Caballete {self.__caballete}")
        self.__patita = False
    else:
        print(f"El caballete ya esta arriba.")
```

Crearemos otra clase la cual representara a un camion de carga, repetimos lo mismo importando el Vehiculo desde el archivo de vehiculo, creamos la clase de camion y llamamos al constructor con los atributos que tenga el vehiculo en general para despues añadir los atributos especificos del camion, siendo su capacidad de carga, el numero de ejes y ademas un contador que tenga la carga actual. Abajo esta definida la carga actual con el fin de realizar pruebas, se utiliza tambien el super() para poder llamar a los atributos de la clase padre y acto seguido definimos los atributos específicos del camión.

```
from vehiculo import Vehiculo

class Camion(Vehiculo):
    def __init__(self, marca: str, modelo: str, anyo: int, capacidadCarga: float, numeroEjes: int, cargaActual: float):
        #llamamos al constructor de la clase padre(Vehiculo)

        self.__cargaActual = 10

        super().__init__(marca, modelo, anyo)

        #atributos especificos del camion.
        self.__capacidadCarga = capacidadCarga
        self.__numeroEjes = numeroEjes
```

Ahora definimos las funciones específicas del camión, siendo este solo la función de cargar y descargar. Se utilizaron algunos if's para poder determinar:

- 1: la capacidad actual para ver si se puede cargar más
- 2: y poder ver si se puede vaciar la carga.

```
def cargar(self):
    print(f"Cargando en el camion...")
    if self.__cargaActual <= 40:
        self.__cargaActual += 1
        print(f"Camion cargado con exito!")
        print(f"Carga actual es de {self.__cargaActual} toneladas")
    else:
        print(f"Carga maxima alcanzada.")

def descargar(self):
    print(f"Descargando el camion...")
    if self.__cargaActual >= 1:
        self.__cargaActual -= 1
        print(f"Carga actual es de {self.__cargaActual} toneladas")
        print(f"Camion descargado con exito!")
    else:
        print(f"Camion completamente vaciado.")
```

Ya estando en la recta final, vamos a crear un archivo llamado “main.py” la cual sera nuestro programa como tal, este se encargara de juntar todos los modulos que hemos creado y los volvera funcionales siendo este quien dara las ordenes de que hacer, comenzamos importando las 3 clases hijas que hemos creado, por ejemplo importamos del archivo motocicleta el vehiculo Motocicleta. (énfasis en la mayuscula).

```
#main.py #recuerda hacer el from e import de cada clase
from motocicleta import Motocicleta
from automovil import Automovil
from camion import Camion
```

Ya habiendo exportado todo definiremos el main() puesto que esta sera nuestra función principal a la hora de trabajar el programa, este es el ejecutable que se usa, tendremos que instanciar el programa. Comenzamos definiendo cada parte con mi “vehiculo” en este caso siendo auto, moto y camión. Después el valor que se le asignara es extraido de los archivos respectivos, dentro de la herencia de cada vehiculo escribiremos en el orden que fue puesto en sus archivos la información que queramos darle a cada vehículo, en este caso pusimos la marca, el modelo, el año, las puertas y su tipo de transmisión. Repetimos con moto y camion poniendo sus respectivos datos.

```
def main():
    print(" || - - - Creando Instancias - - - ||")
    #instanciamos el objeto
    mi_auto = Automovil ("Toyota", "Corolla", 2024, 4, "Automatica")
    mi_moto = Motocicleta ("Harley Davidson", "Softail", 2014, 1450, "si", bool)
    mi_camion = Camion ("Kenworth", "T680", 2022, 40, 3, float)
```


En este punto creamos una forma de mostrar toda la información que haya sido recabada por el programa puesto que desde un principio definimos los valores de cada atributo, este apartado solo arroja todas las funciones sin coherencia alguna con tal de ver que al menos funciona todo. A través de los prints utilizamos todas las funciones de cada clase que hayamos creado y usamos el print con el texto probando métodos para poder diferenciar de qué método estamos recibiendo una respuesta.

```
print("\n || - - - Probando Metodos de mi auto - - - ||")
print(mi_auto.obtenerInformacion())
print(mi_auto.acelerar())
print(mi_auto.frenar())
print(mi_auto.abrirMaletero())
print(mi_auto.ActivarAireAcondicionado())

print("\n || - - - Probando Metodos de mi moto - - - ||")
print(mi_moto.obtenerInformacion())
print(mi_moto.acelerar())
print(mi_moto.frenar())
print(mi_moto.bajarCaballote())
print(mi_moto.subirCaballote())
print(mi_moto.levantarCaballito())

print("\n || - - - Probando Metodos de mi camion - - - ||")
print(mi_camion.obtenerInformacion())
print(mi_camion.acelerar())
print(mi_camion.frenar())
print(mi_camion.cargar())
print(mi_camion.descargar())
```

Como cerecita del pastel, escribiremos esto que nos permitira utilizar el programa.

```
if __name__ == "__main__":
    main()
```

Nuestro producto final se veria algo asi:

```
|| - - - Creando Instancias - - - ||  
  
|| - - - Probando Metodos de mi auto - - - ||  
Vehiculo: Toyota Corolla (2024)  
Acelerando... Velocidad actual: 10.0km/h  
None  
Frenando... Velocidad actual: 0.0km/h  
None  
Abriendo el maletero del auto Vehiculo: Toyota Corolla (2024)....  
None  
Aire acondicionado acivado; que fresco!  
None
```

```
|| - - - Probando Metodos de mi moto - - - ||  
Vehiculo: Harley Davidson Softail (2014)  
Acelerando... Velocidad actual: 10.0km/h  
None  
Frenando... Velocidad actual: 0.0km/h  
None  
Bajando el caballete de la moto...  
Caballete Abajo  
None  
Subiendo el caballete de la moto...  
Caballete Arriba  
None  
Haciendo un caballito en la moto...  
None
```

```
|| - - - Probando Metodos de mi camion - - - ||  
Vehiculo: Kenworth T680 (2022)  
Acelerando... Velocidad actual: 10.0km/h  
None  
Frenando... Velocidad actual: 0.0km/h  
None  
Cargando en el camion...  
Carga maxima alcanzada.  
None  
Descargando el camion...  
Carga actual es de 39 toneladas  
Camion descargado con exito!  
None
```

Conclusiones:

Todo lo aplicado en esta actividad es utilizable en muchos otros trabajos que tengan que hacerse a futuro puesto que usa las bases de la programación orientada a objetos, la escalabilidad, la permutación y la modularidad son cosas que caracteriza mucho a la programación orientada a objetos, realmente ayuda mucho el tener un diagrama previamente hecho que ayude a orientarte o a ser tu guía a lo largo de todo el proyecto.

Fue bastante entretenido hacer este programa, le veo mucho futuro a las cosas que podre hacer siguiendo esta forma de hacer los programas, el hecho de ir quitando poquito a poco las cosas que tengan en común las clases lo vuelve más dinámico, aunque a la par lo vuelve un poco mas revoltoso. Más no obstante, una vez logras comprenderlo ahorra muchos procesos a la hora de trabajar, estamos quitando todo aquello que sea redundante al programar y solo hacemos que las cosas se ajusten a nuestras necesidades.